

How the AI Agents & Customer Risk Summary Actually Work

The rules, thresholds, and formulas behind the Upsell, Retention, Revenue, and Follow-up agents, and behind every number shown on a customer's risk summary — churn probability, failure probability, and the proposed next step.

Scope: apps/crm-service (4 rule-based + LLM agents) and the customer status-summary endpoint

LLM provider: Google Gemini via @google/genai (migrated from OpenAI)

Generated: July 2026

Contents

1. Shared architecture: Rule Engine → Context Builder → LLM → Validation

2. Upsell Agent

3. Retention Agent

4. Revenue Agent

5. Follow-up Agent

6. Customer Risk Summary — churn & failure classification

7. Proposed Next Step — the decision tree

8. Retention Suggestion score (shown alongside the action)

9. The Reasoning tab — how it explains itself

10. The Gemini LLM layer

1 Shared architecture

Every one of the four agents (Upsell, Retention, Revenue, Follow-up) is built from the same four-stage pipeline, always run in this fixed order:



- **RULE ENGINE** is the only layer allowed to decide *whether* an opportunity/risk exists and *which category* it falls into. It runs safety gates first (automatic follow-up must be on, contact-attempt limits not exceeded, a phone/email must exist) and then a fixed cascade of business rules, first match wins.
- **CONTEXT BUILDER** translates raw database rows into a clean, LLM-safe profile object — the LLM never sees a raw Prisma row.
- **LLM CLIENT** (Gemini, model `gemini-2.5-flash` by default) is only allowed to pick the specific offer, channel, message, priority, and confidence — never the category, never a dollar figure. If `GEMINI_API_KEY` is unset, times out (8s), or returns malformed JSON, the client returns `null` and never throws.
- **VALIDATION** re-checks every field the LLM produced against ground truth and company policy before anything is allowed to reach a customer. It can reject the LLM's output but can never let the LLM override the rule engine's category.

⚠ IMPORTANT ASYMMETRY BETWEEN THE FOUR AGENTS

Upsell, Retention, and Revenue all require a successful Gemini response to surface any positive action. Their validation services contain the line `finalCategory = !blocked && llmRec ? ruleCategory : 'no_<x>'` — if the LLM call fails for any reason, the result silently becomes "no upsell" / "no action" / "no opportunity", *even if the rule engine found a completely valid, real opportunity*. The rule engine's category is never surfaced on its own.

Follow-up is the exception — its final `action` comes directly from the rule engine regardless of the LLM, and only the message text falls back to canned copy if Gemini is unavailable. It degrades gracefully; the other three do not.

Practically: with no `GEMINI_API_KEY` configured, Upsell/Retention/Revenue currently produce "no action" for every customer, while Follow-up and the churn/failure risk classification (section 6) are fully rule-based and unaffected.

2 Upsell Agent

```
apps/crm-service/src/upsell/*, apps/crm-service/src/agents/... (UpsellAgentService)
```

Rule engine — `UpsellRuleEngine.evaluate()`

Safety gates (any one blocks the customer entirely): follow-up disabled · 3+ previous upsell attempts · no phone/email on file.

Then, first match wins:

#	Condition	Category	Priority
1	Oldest equipment > 8 years old	replacement	High
2	≥3 repairs in the last 12 months	maintenance_plan	High
3	New equipment installed AND warranty still active	extended_warranty	Normal
4	No service in > 180 days	preventive_service	Normal
5	Premium segment AND annual spend > \$3,600	premium_upgrade	Normal
—	None of the above	no_upsell	—

What the LLM is asked

The context builder sends: customer segment, equipment list (type/age/warranty status), maintenance agreement status, repair count, days since last service, annual spend, previous attempt count, preferred channel, recent quotes, notes, plus the rule engine's category/reason. The LLM returns `offer`, `bundle`, `priority` (Low/Medium/High), `channel`, `reason`, `message` (≤320 chars), `confidence` (0–1) as strict JSON.

Validation gates (blocking)

`OPTED_OUT` · `ATTEMPT_LIMIT_EXCEEDED` (≥ 3) · `NO_CONTACT_CHANNEL` · `DUPLICATE_OFFER` (same category sent in last 30 days) · `CATEGORY_NOT_AVAILABLE` · `NO_DELIVERABLE_CHANNEL` . Priority resolves from the LLM's value when present, otherwise falls back to the rule engine's `highPriority` flag (high/medium).

Caching & triggers

Recommendations are persisted to `upsell_recommendations` and reused for **24 hours** before regenerating. Triggered by: service completion, customer profile update, a daily batch job, or a manual request from the dashboard.

FALLBACK USED ONLY IF THE AGENT RECORD CAN'T BE LOADED (CUSTOMER LOOKUP RACE)

A separate, simpler weighted-scoring calculation runs entirely inside `CustomersService` — *not* the rule engine above:

```
base weights: maintenance_plan=0.25  replacement=0.20  service=0.20
// boosts, all additive:
equipment age > 8y          → replacement      += 0.45
failure history > 2         → maintenance_plan += 0.45
days since last service > 180 → service        += 0.40
failure risk ≥ 50%          → maintenance_plan += 0.15 (+0.10 replacement if also >8y)
churn risk ≥ 50%            → maintenance_plan += 0.10, service += 0.10
avg monthly spend ≥ $250    → maintenance_plan += 0.08
// normalize all three scores to sum to 1.0, highest wins
```

3 Retention Agent

```
apps/crm-service/src/retention/*, apps/crm-service/src/agents/retention.agent.ts
```

Rule engine — `RetentionRuleEngine.evaluate()`

Safety gates: follow-up disabled · 3+ previous retention attempts · no phone/email on file.

#	Condition	Reason code	Priority
1	≥2 low ratings/complaints on file	CUSTOMER_COMPLAINTS	High
2	Maintenance agreement expired	AGREEMENT_EXPIRED	Normal
3	≥3 repairs in the last 12 months	FREQUENT_REPAIRS	Normal
4	Premium segment, annual spend > \$2,400 , AND engagement trend is decreasing	HIGH_VALUE_CUSTOMER	Normal
5	No service in > 180 days	CUSTOMER_INACTIVE	Normal
—	None of the above	NONE	—

Complaints are checked **first**, ahead of agreement/repair/value/inactivity signals — dissatisfaction risk outranks everything else in this cascade.

What the LLM is asked

Segment, annual spend, engagement trend, agreement status/expiry, days since service, repair count, complaint count, previous attempts, active contract count. Returns `strategy`, `action` (must be one of `premium_contract_offer` / `discount_retention_offer` / `maintenance_plan_offer` / `no_action`), `channel`, `priority`, `reason`, `message`, `confidence`.

Validation & offer policy

Same blocking-check set as Upsell. Company policy fixes the offer terms per action — the LLM cannot invent a discount amount:

Action	Offer
premium_contract_offer	Premium, 0% discount
discount_retention_offer	20% discount
maintenance_plan_offer	Standard, 10% discount
no_action	None

Caching

Persisted to `retention_recommendations`, reused for 24 hours. This table also tracks real outcomes (`retentionSuccess`, `revenueGenerated`, `respondedAt`) — the only agent whose historical performance is fed back into anything downstream (used to calibrate the analytics-service revenue-recommendations panel; see the separate AI Revenue Recommendations documentation).

4 Revenue Agent

```
apps/crm-service/src/revenue/*, apps/crm-service/src/agents/revenue.agent.ts
```

Rule engine — `RevenueRuleEngine.evaluate()`

Same safety gates as the others. Ordered by **business priority** — **cash-at-risk first, win-back last**:

#	Condition	Category	Dollar impact used
1	Any invoice overdue > 30 days (worst one picked)	payment_collection	That invoice's balance due
2	Any quote pending > 7 days with no response (oldest picked)	quote_recovery	That quote's total
3	Agreement expiring within 30 days , or status PENDING_RENEWAL	agreement_renewal	—
4	No active agreement AND oldest equipment > 8 years	maintenance_plan	—
5	No service in > 180 days	re_engagement	—
—	None of the above	no_opportunity	—

The dollar impact figure is **always computed by the rule engine from the actual invoice/quote row**, never by the LLM — the LLM's `expectedImpact` field is a text framing only (e.g. "recovers overdue cash flow"), not a number.

What the LLM is asked

Segment (derived from lifetime spend: premium $\geq$ \$2,400/yr, standard $\geq$ \$600/yr, else budget), lifetime spend, equipment ages, open quotes, overdue invoices, days since last service, agreement status/expiry. Returns `strategy`, `recommendedAction`, `priority`, `expectedImpact` (string), `channel`, `reason`, `message`, `confidence`.

Caching

Persisted to `revenue_recommendations`, 24-hour TTL. Powers the `revenueRecommendation` field on the customer status-summary.

5 Follow-up Agent

```
apps/crm-service/src/followup/*
```

Rule engine — `FollowupRuleEngine.evaluate()`

Safety gates: follow-up disabled · >5 previous attempts · no phone/email.

#	Condition	Action
1	Lead only — status NEW/CONTACTED/QUALIFIED and > 3 days old	LEAD_FOLLOWUP (cold lead)
2	Quote SENT/VIEWED and > 3 days with no response	QUOTE_FOLLOWUP
3	Maintenance agreement expired	RETENTION
4	Engagement status INACTIVE, or no service in > 90 days	REENGAGEMENT
—	None of the above	no follow-up needed

THIS IS THE ONE AGENT THAT DEGRADES GRACEFULLY

The final `action` in the returned decision comes **directly from the rule engine**, not from validation or the LLM. If Gemini is unavailable, the follow-up still happens — only the message text falls back from an LLM-written one to a canned template, and the channel falls back to whichever of phone/email is actually on file. Contrast this with Upsell/Retention/Revenue in section 1.

Validation gates

`NO_CONTACT_CHANNEL` · `ATTEMPT_LIMIT_EXCEEDED (>5)` · `OPTED_OUT` · `INTERVAL_NOT_SATISFIED` (a follow-up already went out within the last 48h) · `CHANNEL_UNAVAILABLE` · `followupWithin` ("Today" / "Within 2 days" / "This week") maps to a concrete scheduled timestamp.

Note: unlike the other three agents, a null/blocked result here means `decide()` returns `null` outright — there is no persisted "no action" record.

6 Customer Risk Summary — churn & failure classification

`apps/crm-service/src/customers/customers.service.ts` — `getStatusSummary()`

This is the popup/sidebar shown from the Customers list. Churn and failure risk are **pure arithmetic on this customer's own CRM data** — there is no ML model call and no LLM call anywhere in this calculation. (An earlier version called a Python churn-service with scikit-learn/XGBoost models; that call was removed entirely and replaced with the rule-based formula below.)

Churn probability — `classifyChurnAndFailureRisk()`

```
churn = bucket(days_since_last_service)
  + (service_count_last_year == 0 ? 0.25 : 0)
  + (customer_tenure_days < 90 ? 0.08 : 0)
  capped at 0.95

bucket(days_since_last_service):
  > 180 days → 0.45
  > 90 days  → 0.25
  otherwise  → 0.08
```

Failure (equipment breakdown) probability

```
failure = bucket(equipment_age_days)
          + (days_since_last_service > 180 ? 0.2 : 0)
          + min(0.2, failure_history_count × 0.08)
          capped at 0.95

bucket(equipment_age_days):
  > 10 years → 0.45
  > 5 years  → 0.25
  otherwise → 0.08

failure_history_count = number of reviews with rating ≤ 2
```

Risk level bands (applies to both churn and failure)

Probability	Level
≥ 70%	High
40% – 69%	Medium
< 40%	Low

Current status text

Condition	Displayed status
Customer record inactive	"Inactive customer record"
Engagement status is INACTIVE	"Inactive engagement"
> 180 days since service	"No confirmed service for N days"
> 90 days since service	"Service is overdue by N days"
Follow-up paused	"Active, automatic follow-up paused"
Otherwise	"Active, last confirmed service N days ago"

7 Proposed Next Step — the decision tree

Built from the churn/failure numbers above plus a `recommended_action` classification (URGENT_INTERVENTION / RETENTION / MAINTENANCE / REENGAGEMENT / NONE), evaluated first match wins:

#	Condition	Next step shown
1	Automatic follow-up is paused	"Review manually because automatic follow-up is paused."
2	Churn > 70% and failure > 70%	"Call today, schedule maintenance, and assign a retention owner."
3	Churn > 70%	"Send a retention offer and book a check-in call."
4	Failure > 70%	"Schedule preventive maintenance before the next breakdown risk window."
5	Recommended action is REENGAGEMENT, or > 90 days since service	"Send a re-engagement message and offer a service slot."
6	Churn ≥ 40% or failure ≥ 40%	"Monitor this week and prepare a targeted follow-up."
—	None of the above	"No immediate action. Keep standard follow-up cadence."

The `recommended_action` classification itself: churn>70% *and* failure>70% → URGENT_INTERVENTION; churn>70% → RETENTION; failure>70% → MAINTENANCE; churn>45% → REENGAGEMENT; else → NONE.

8 Retention Suggestion score (shown on the risk summary)

The Retention Agent (section 3) decides the *action*. Three extra numbers are shown alongside it that are **informational context, not inputs to that decision**:

```
pConvert      = this customer's upsell confidence (section 2)
LTV           = average monthly spend × 12
score         = pConvert × LTV × (1 – churn probability)
priority (if no agent record yet) = score > 1500 ? high : score ≥ 500 ? medium : low
```

If a real Retention Agent record exists, its `action` / `priority` / `channel` / `reason` are used as-is; `pConvert` / `LTV` / `score` are always the formula above regardless. `triggerImmediately` is set whenever the customer has 3+ complaints on file — independent of what action was actually decided, so it can appear even next to a "No Action" result.

9 The Reasoning tab — how it explains itself

Every card on the Reasoning tab (Customer Details → Reasoning) shows three panels, each populated with this specific customer's real numbers — not generic template text:

Panel	Content
Rule thresholds	The general thresholds for that decision (same for every customer) — e.g. the Upsell cascade table in section 2.
Calculation for this customer	The actual arithmetic worked out with this customer's real inputs plugged in — e.g. "52 days since last service ($\leq 90d$, $+0.08$) + 0 services in the last year ($+0.25$) + 45 days as a customer ($< 90d$, $+0.08$) = 16% → Low".
Why	A plain-language interpretation tied to that specific result.

For the Upsell and Retention panels specifically, the "calculation" text includes the rule engine's own per-customer match reason (e.g. "Customer has 3 low ratings or complaints on file"), threaded through from the agent's stored decision — the same string that decided the category in the first place, not a re-derived summary.

10 The Gemini LLM layer

All four agents (and the separate analytics-service revenue-recommendations panel) share the same client shape, using `@google/genai` :

- No API key → every LLM call returns `null` immediately, no network request made.
- Model: `GEMINI_MODEL_<AGENT>` env var, default `gemini-2.5-flash` for all five call sites.
- `responseMimeType: "application/json"` — the model is constrained to return valid JSON.
- 8-second timeout, enforced both via the SDK's `httpOptions.timeout` and a manual `Promise.race` guard (the SDK-level timeout is known to be unreliable on some versions).
- Every field of the response is type/enum/length-validated before being trusted; any mismatch is treated identically to a total failure — returns `null`.
- The client never throws — any error (missing key, timeout, malformed JSON, invalid shape) is caught and logged as a warning, then treated as "no LLM recommendation available."

Generated from the current source of `apps/crm-service` and `apps/admin-dashboard`. Thresholds and formulas quoted here are read directly from the rule-engine source files as of this report's generation date — if those files change, this document will drift and should be regenerated.